TASKUP: GERENCIADOR DE TAREFAS

Sabrina Rayssa Teixeira
Prof. Ricardo Bicalho

RESUMO

Este artigo apresenta o TaskUp, um gerenciador de tarefas pessoais formado por uma API RESTful e por uma interface que a consome. O objetivo do trabalho é praticar desenvolvimento backend, modelagem de banco de dados e criação de APIs REST em um sistema pequeno. A API foi feita em Node.js com Express e usa o SQLite como banco de dados. Com ela é possível cadastrar, listar, consultar, editar e excluir tarefas. O backend, o banco e a interface web já estão integrados: a interface consome a API para cadastrar usuários, organizar tarefas em categorias, criar checklists de subtarefas, visualizar os detalhes e as subtarefas diretamente na lista, buscar e filtrar tarefas, e listar, criar, editar, concluir e excluir tarefas. A exclusão funciona como uma lixeira, permitindo restaurar a tarefa antes de apagá-la definitivamente. A publicação em nuvem fica para a próxima etapa.

Palavras-chave: API REST; Node.js; Express; SQLite; gerenciamento de tarefas.

ABSTRACT

This paper presents TaskUp, a personal task manager made of a RESTful API and an interface that uses it. The goal of the work is to practice backend development, database modeling and REST API design in a small system. The API was built with Node.js and Express and uses SQLite as its database. It allows users to create, list, view, edit and delete tasks. The backend, the database and the web interface are now integrated: the interface uses the API to register users, organize tasks into categories, create subtask checklists, view task details and subtasks directly in the list, search and filter tasks, and list, create, edit, complete and delete tasks. Deletion works as a trash bin, so a task can be restored before being permanently removed. Cloud deployment is left for the next stage.

Keywords: REST API; Node.js; Express; SQLite; task management.

1. INTRODUÇÃO

Um gerenciador de tarefas é uma aplicação fácil de entender e boa para praticar os conceitos da disciplina. O cliente, que é a interface, fica separado do servidor, que guarda os dados e aplica as regras, e os dois conversam por meio de uma API. Assim, a mesma API pode atender uma interface web ou um aplicativo móvel.

O TaskUp foi desenvolvido na unidade curricular Sistemas Distribuídos e Mobile. O objetivo é criar uma aplicação para organizar tarefas e, com ela, aplicar conceitos de backend, banco de dados e APIs RESTful. Como metas, o trabalho definiu implementar as operações básicas sobre as tarefas, integrar o backend a uma interface web e acrescentar recursos de organização, como categorias, subtarefas, prazos, prioridades, filtros e uma lixeira para exclusões reversíveis.

2. FUNDAMENTAÇÃO TEÓRICA E TECNOLOGIAS

2.1. API RESTful

REST é um estilo de arquitetura apresentado por Fielding (2000). Nele, cada coisa que a aplicação manipula é um recurso com um endereço (URL), e as operações usam os métodos do HTTP: GET para consultar, POST para criar, PUT para alterar e DELETE para excluir. A resposta traz um código de status, como 200 (sucesso), 201 (criado), 204 (sem conteúdo), 400 (requisição inválida), 404 (não encontrado) e 409 (conflito). No TaskUp, os recursos principais são usuários, categorias, tarefas e subtarefas, acessados por rotas como `/usuarios`, `/categoria` e `/tarefas`.

2.2. Node.js, Express e SQLite

Foram usados o Node.js, que executa JavaScript no servidor (NODE.JS, [s.d.]), e o Express, um framework que simplifica a criação de rotas (EXPRESS, [s.d.]). O banco de dados é o SQLite, que guarda tudo em um único arquivo e não exige instalar um servidor de banco (SQLITE, [s.d.]). Ele é acessado pelo módulo `node:sqlite`, disponível a partir do Node.js 22.5. Por isso, as únicas dependências do projeto são o Express e o CORS, este último para liberar o acesso à API a partir de outra origem, como o frontend. O projeto exige Node.js 22.5 ou superior.

3. MODELAGEM E IMPLEMENTAÇÃO

3.1. Funcionalidades

As funcionalidades do TaskUp estão na Tabela 1 e são atendidas pelos endpoints da Tabela 2. O sistema foi pensado para ser simples de usar: cada tarefa pertence a um usuário, pode ter uma categoria, um prazo, uma prioridade e um checklist de subtarefas. A categoria é opcional; quando uma categoria é excluída, as tarefas continuam existindo, mas ficam sem categoria. A busca ignora diferenças de maiúsculas, minúsculas e acentos, e os filtros de status, categoria e busca podem ser combinados. No celular, a navegação principal passa para uma barra inferior, facilitando o uso em telas pequenas.

Além disso, ao clicar em um cartão na lista, a interface expande a descrição, a prioridade, o prazo e as subtarefas da tarefa. Nesse painel, o usuário pode marcar ou desmarcar uma subtarefa e acompanhar o progresso do checklist sem abrir outra janela. O botão “Subtarefas” continua disponível para adicionar ou remover itens.

Tabela 1 – Funcionalidades do TaskUp.

Código	Funcionalidade	Descrição
RF01	Cadastrar tarefa	Cria uma tarefa com título obrigatório e descrição, prazo, prioridade e categoria opcionais. A tarefa nasce com status pendente.
RF02	Listar tarefas	Retorna as tarefas ativas do usuário, das mais recentes para as mais antigas.
RF03	Visualizar detalhes	Retorna os dados de uma tarefa específica.
RF04	Editar tarefa	Altera título, descrição, prazo, prioridade, categoria ou status.
RF05	Marcar como concluída	Muda o status para concluída e permite reabrir a tarefa.
RF06	Excluir tarefa	Move a tarefa para a lixeira, sem apagá-la imediatamente.
RF07	Restaurar ou excluir definitivamente	Na lixeira, permite restaurar a tarefa ou apagá-la de vez. A exclusão definitiva apaga as subtarefas em cascata.
RF08	Gerenciar categorias	Permite cadastrar, listar, renomear e excluir categorias, associar tarefas a uma categoria e filtrar a listagem por categoria. A categoria é opcional; ao excluí-la, as tarefas ficam sem categoria.
RF09	Gerenciar subtarefas	Permite adicionar, concluir e remover subtarefas de uma tarefa, formando um checklist com progresso.
RF10	Filtrar e buscar tarefas	Permite filtrar por status e por categoria e buscar pelo título sem diferenciar maiúsculas ou acentos. Os filtros podem ser combinados.
RF11	Visualizar detalhes na lista	Ao clicar no cartão, exibe descrição, prioridade, prazo e subtarefas, permitindo concluir itens do checklist sem abrir outra janela.

Fonte: Elaborado pelos autores (2026).

3.2. Modelo de dados

O banco possui quatro tabelas relacionadas. `usuarios` armazena nome, telefone e data de criação, com chave primária composta. `categorias` pertence a um usuário e armazena o nome da categoria. `tarefas` armazena título, descrição opcional, prazo opcional, prioridade, status e data de criação, além de referenciar o usuário por nome e telefone e, opcionalmente, uma categoria. `subtarefas` armazena o título, o indicador de concluída e a data de criação de cada item do checklist, referenciando a tarefa. A coluna `excluida_em` implementa a lixeira: quando está vazia, a tarefa aparece na listagem normal; quando recebe uma data, a tarefa passa a aparecer apenas na lixeira. Quando uma categoria é excluída, as tarefas ligadas a ela ficam sem categoria. Quando uma tarefa é apagada definitivamente, suas subtarefas são apagadas em cascata; enquanto a tarefa está na lixeira, as subtarefas são preservadas para a restauração.

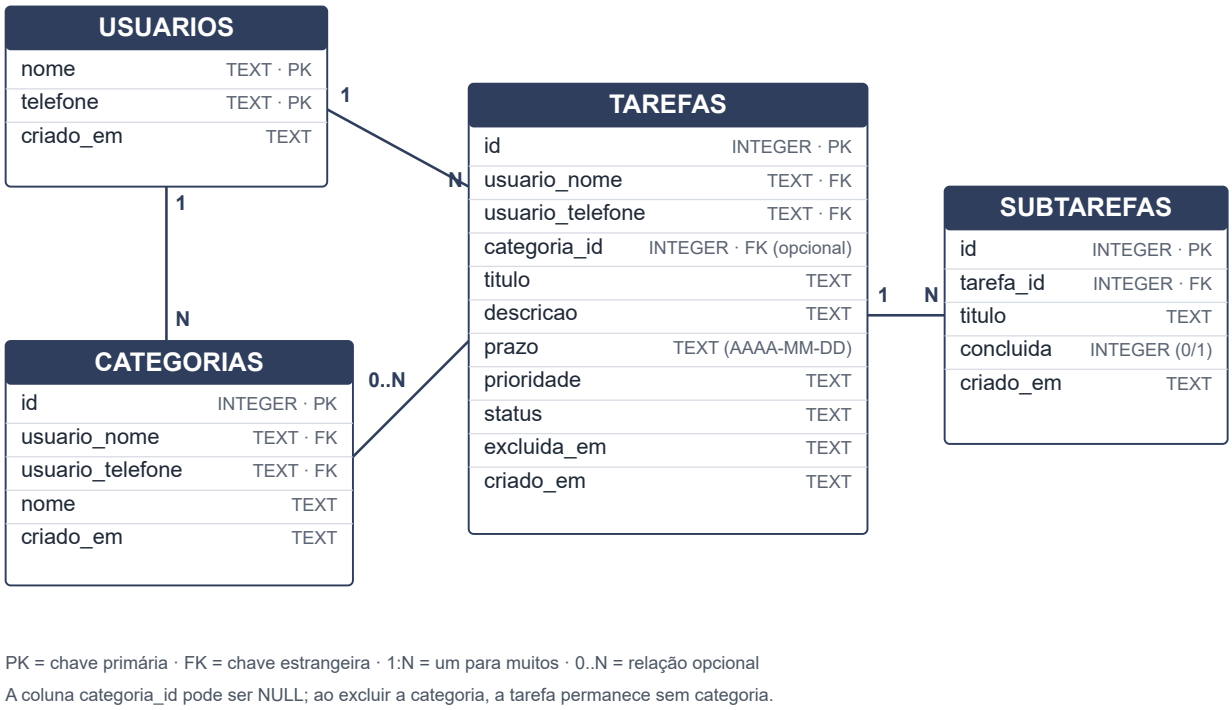


Figura 1 – Diagrama entidade-relacionamento do TaskUp.
Fonte: Elaborado pelos autores (2026).

As relações são: um usuário possui várias tarefas (1:N) e várias categorias (1:N); uma categoria pode estar em várias tarefas (1:N), e a tarefa pode não ter categoria; uma tarefa possui várias subtarefas (1:N), apagadas em cascata quando a tarefa é excluída definitivamente.

3.3. API do backend

O arquivo `db.js` cria as tabelas e aplica pequenas migrações; o `server.js` apenas registra os routers. Cada domínio tem sua própria pasta em `src`, com as consultas SQL (`sql.js`) separadas das rotas (`routes.js`). A Tabela 2 resume os endpoints.

Tabela 2 – Endpoints principais da API.

Método	Rota	Função	Respostas
GET	`/usuarios`	Lista usuários	200
POST	`/usuarios`	Cadastra usuário	201, 400, 409
GET	`/categoria`	Lista categorias	200, 400
POST	`/categoria`	Cadastra categoria	201, 400, 404, 409
PUT	`/categoria/:id`	Edita categoria	200, 400, 404, 409
DELETE	`/categoria/:id`	Exclui categoria	204, 404
GET	`/tarefas`	Lista tarefas ativas	200, 400
GET	`/tarefas/lixeira`	Lista tarefas da lixeira	200, 400
GET	`/tarefas/:id`	Detalha tarefa	200, 404
POST	`/tarefas`	Cadastra tarefa	201, 400, 404
PUT	`/tarefas/:id`	Edita tarefa e status	200, 400, 404
PUT	`/tarefas/:id/restaurar`	Restaura tarefa da lixeira	200, 400, 404
DELETE	`/tarefas/:id`	Move a tarefa para a lixeira	204, 404
DELETE	`/tarefas/:id/definitivo`	Apaga a tarefa do banco	204, 400, 404
GET	`/tarefas/:id/subtarefas`	Lista as subtarefas da tarefa	200, 404
POST	`/tarefas/:id/subtarefas`	Cadastra uma subtarefa	201, 400, 404
PUT	`/tarefas/:id/subtarefas/:subtarefaId`	Edita ou conclui uma subtarefa	200, 400, 404
DELETE	`/tarefas/:id/subtarefas/:subtarefaId`	Exclui uma subtarefa	204, 400, 404

Fonte: Elaborado pelos autores (2026).

O servidor valida o título obrigatório, o status (`pendente` ou `concluida`), a prioridade (`baixa`, `media` ou `alta`) e o formato do prazo (`AAAA-MM-DD`). A categoria é opcional, mas quando informada precisa pertencer ao mesmo usuário da tarefa. O título da subtarefa é obrigatório e a subtarefa precisa pertencer à tarefa informada na rota. Tarefas que estão na lixeira não aceitam alterações nas subtarefas: é preciso restaurá-las antes. A rota de exclusão definitiva só aceita tarefas que já estão na lixeira.

3.4. Interface web

A interface foi feita com HTML, CSS e JavaScript puro, sem framework. O arquivo `config.js` define o endereço da API; `script.js` concentra o estado da aplicação, as chamadas à API e a renderização das telas; `index.html` organiza as views e os modais; `style.css` cuida do layout e da responsividade. Há três views principais: Minhas tarefas, Concluídas e Lixeira. Os modais permitem criar e editar tarefas, gerenciar categorias e adicionar ou remover subtarefas. O login é feito com nome e telefone, e o usuário fica salvo no navegador para as próximas visitas. Ao trocar de view, a página volta ao topo para mostrar o resumo geral.

4. RESULTADOS PARCIAIS

4.1. Teste do backend e da interface

O backend e a interface foram executados localmente e testados com requisições HTTP e pelo navegador. O cadastro de usuário retornou 201 e um cadastro repetido retornou 409. O cadastro de categoria retornou 201, uma categoria repetida retornou 409 e a edição retornou 200. A criação de tarefa retornou 201 com status pendente, prioridade, prazo e categoria; uma prioridade inválida, um prazo fora do formato e uma categoria de outro usuário retornaram 400. O cadastro de subtarefa retornou 201, a conclusão retornou 200 e a listagem de tarefas passou a mostrar o progresso do checklist. Criar uma subtarefa sem título retornou 400, assim como tentar alterar subtarefas de uma tarefa na lixeira. A conclusão da tarefa retornou 200 e atualizou o status. A exclusão retornou 204, a tarefa saiu da listagem normal e passou a aparecer em `/tarefas/lixeira`. A restauração retornou 200 e a tarefa voltou para a listagem com suas subtarefas. A exclusão definitiva retornou 204 somente para tarefas na lixeira e apagou as subtarefas em cascata. Uma consulta a uma tarefa apagada retornou 404.

Na interface, foram testados login, gerenciamento de categorias, associação e filtro por categoria, checklist de subtarefas com progresso, busca por título sem diferenciar maiúsculas ou acentos, filtro por status, combinação de filtros, busca na lixeira, criação de categoria pelo formulário de tarefa, navegação pelo menu inferior em tela pequena, criação, edição, conclusão, envio para a lixeira, restauração, exclusão definitiva, persistência após sair e entrar, limpeza dos filtros ao sair e saída do usuário. Também foi testado o clique no cartão da tarefa, que expande a descrição, a prioridade, o prazo e o checklist de subtarefas na própria lista; uma subtarefa pode ser concluída nesse painel e o progresso é atualizado sem fechar o cartão. Ao trocar de tela depois de rolar a lista, a página volta ao topo para mostrar o resumo geral. A Tabela 3 resume os principais testes.

Tabela 3 – Principais testes realizados.

Teste	Resultado esperado	Resultado obtido
Cadastrar usuário novo	201	201
Cadastrar usuário repetido	409	409
Criar tarefa com prazo, prioridade e categoria	201	201
Criar tarefa com prioridade inválida	400	400
Criar tarefa com categoria de outro usuário	400	400
Cadastrar categoria	201	201
Cadastrar categoria repetida	409	409
Criar subtarefa	201	201
Criar subtarefa sem título	400	400
Concluir subtarefa	200 e `concluida = 1`	200
Alterar subtarefa de tarefa na lixeira	400	400
Concluir tarefa	200 e status `concluida`	200
Mover tarefa para a lixeira	204	204
Restaurar tarefa	200	200
Excluir definitivamente tarefa ativa	400	400
Excluir definitivamente tarefa na lixeira	204	204
Consultar tarefa apagada	404	404
Buscar ignorando acentos e maiúsculas	Encontra a tarefa	Funcionou
Combinar busca, status e categoria	Somente tarefas que atendem a todos os filtros	Funcionou
Expandir tarefa na lista	Descrição, prioridade, prazo e subtarefas aparecem no cartão	Funcionou
Concluir subtarefa na lista	Progresso atualizado sem fechar o cartão	Funcionou
Navegar em tela pequena	Menu inferior acessa as telas	Funcionou
Fluxo completo na interface	Sem erros	Sem erros

Fonte: Elaborado pelos autores (2026).

4.2. Próximas etapas

Falta publicar o backend e o frontend em um serviço de nuvem. Como evolução do sistema, estão previstas notificações de prazos.

5. CONSIDERAÇÕES FINAIS

Nesta etapa, a API do TaskUp permite cadastrar e consultar usuários e categorias e executar operações CRUD em tarefas e subtarefas, com lixeira, restauração, organização por categorias e checklists. O banco valida os dados e vincula cada tarefa a um usuário. A interface web já consome a API e permite gerenciar categorias, buscar e filtrar tarefas e acompanhar o progresso das subtarefas. As próximas etapas são publicar o sistema em nuvem e implementar notificações de prazos.

6. REFERÊNCIAS

CORS. *Cross-Origin Resource Sharing*. MDN Web Docs. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/HTTP/CORS. Acesso em: 3 nov. 2026.

EXPRESS. *Express: Node.js web application framework*. [S.l.: s.n., s.d.]. Disponível em: https://expressjs.com/. Acesso em: 3 nov. 2026.

FIELDING, Roy Thomas. *Architectural styles and the design of network-based software architectures*. 2000. Tese (Doutorado) – University of California, Irvine, 2000.

NODE.JS. *Node.js documentation*. [S.l.: s.n., s.d.]. Disponível em: https://nodejs.org/docs/latest-v22.x/api/. Acesso em: 3 nov. 2026.

SQLITE. *SQLite documentation*. [S.l.: s.n., s.d.]. Disponível em: https://www.sqlite.org/docs.html. Acesso em: 3 nov. 2026.